Component Interaction Diagrams

Detailed Mermaid diagrams documenting recmeet's architecture, component interactions, and internal control flow. These diagrams reflect the actual source code implementation, not aspirational design.

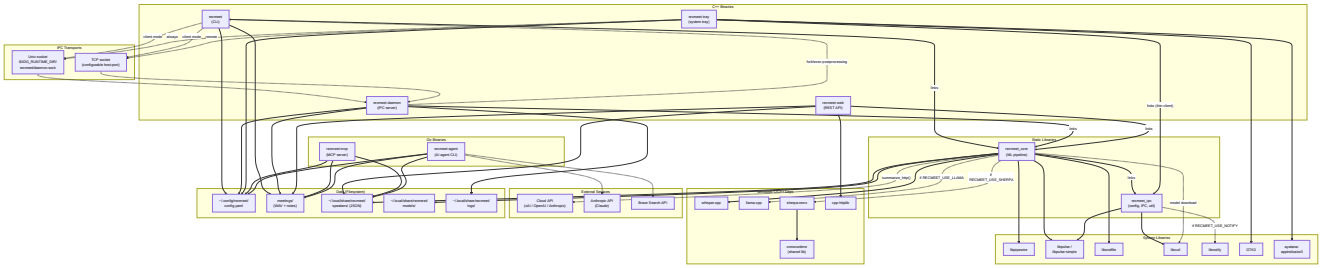
Table of Contents

1. Top-Level Component Interaction
2. Build Topology and Library Dependencies
3. Daemon Internals
4. IPC Server Poll Loop
5. IPC Client Flow
6. IPC Protocol Wire Format
7. Recording Pipeline
8. Postprocessing Pipeline
9. Tray Applet
10. CLI Mode Selection
11. Subprocess Postprocessing
12. Audio Capture Subsystem
13. Go Tools Module

---

1. Top-Level Component Interaction

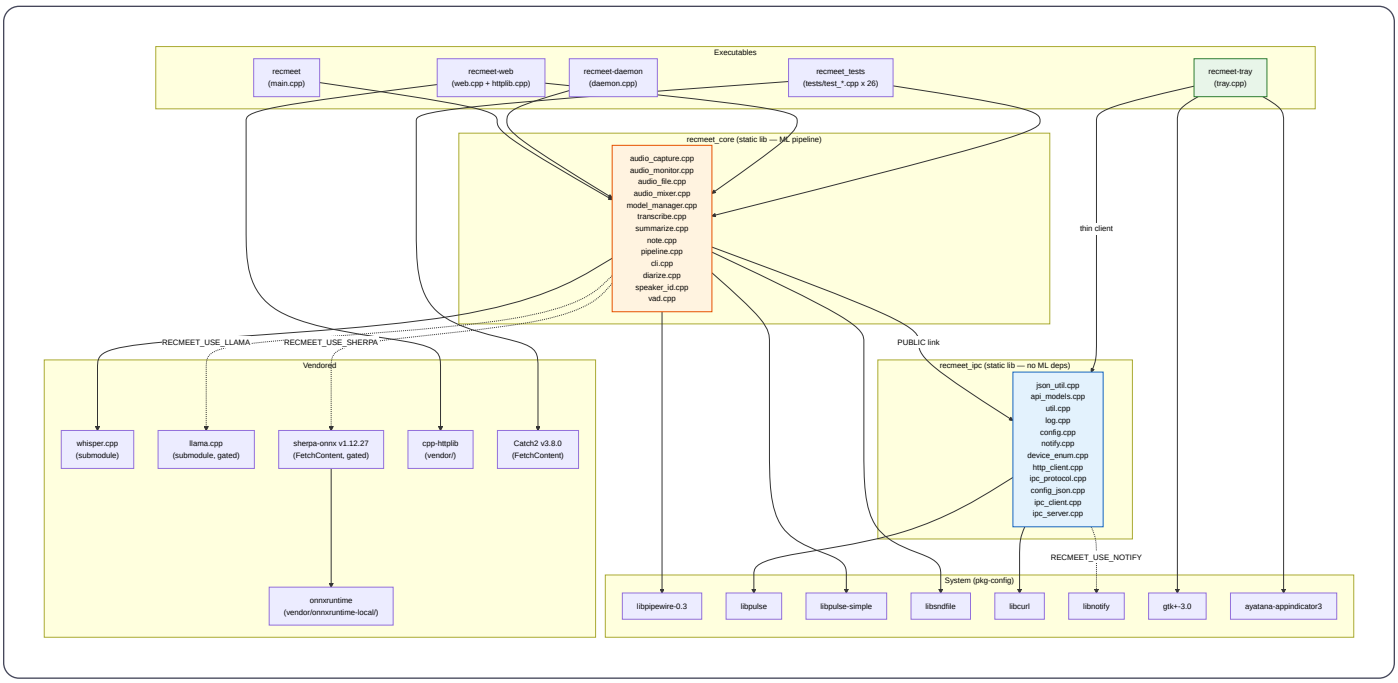
All runtime communication paths between binaries, libraries, and external systems. Solid lines are compile-time links; dashed lines are runtime IPC or network calls.



---

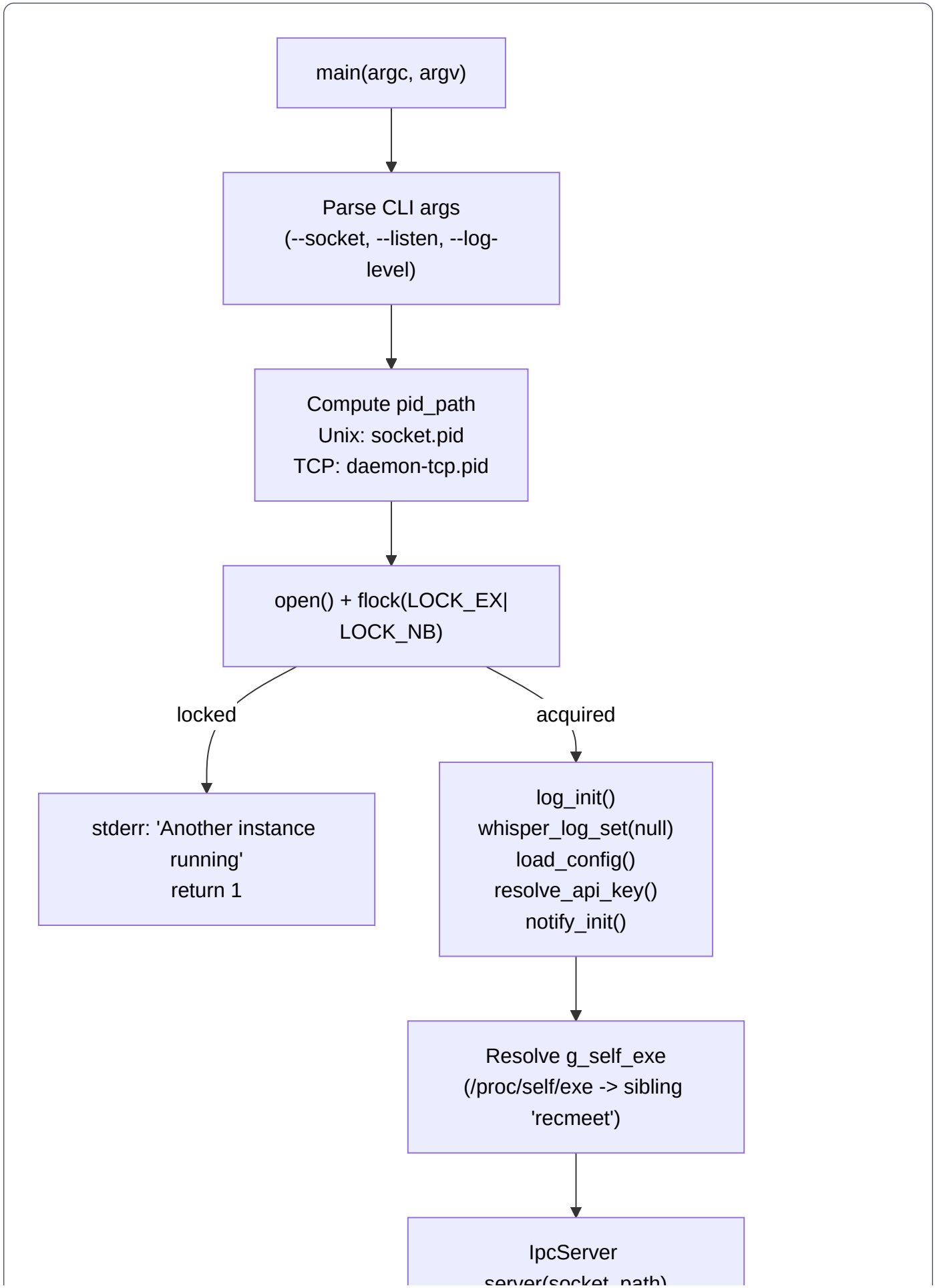
2. Build Topology and Library Dependencies

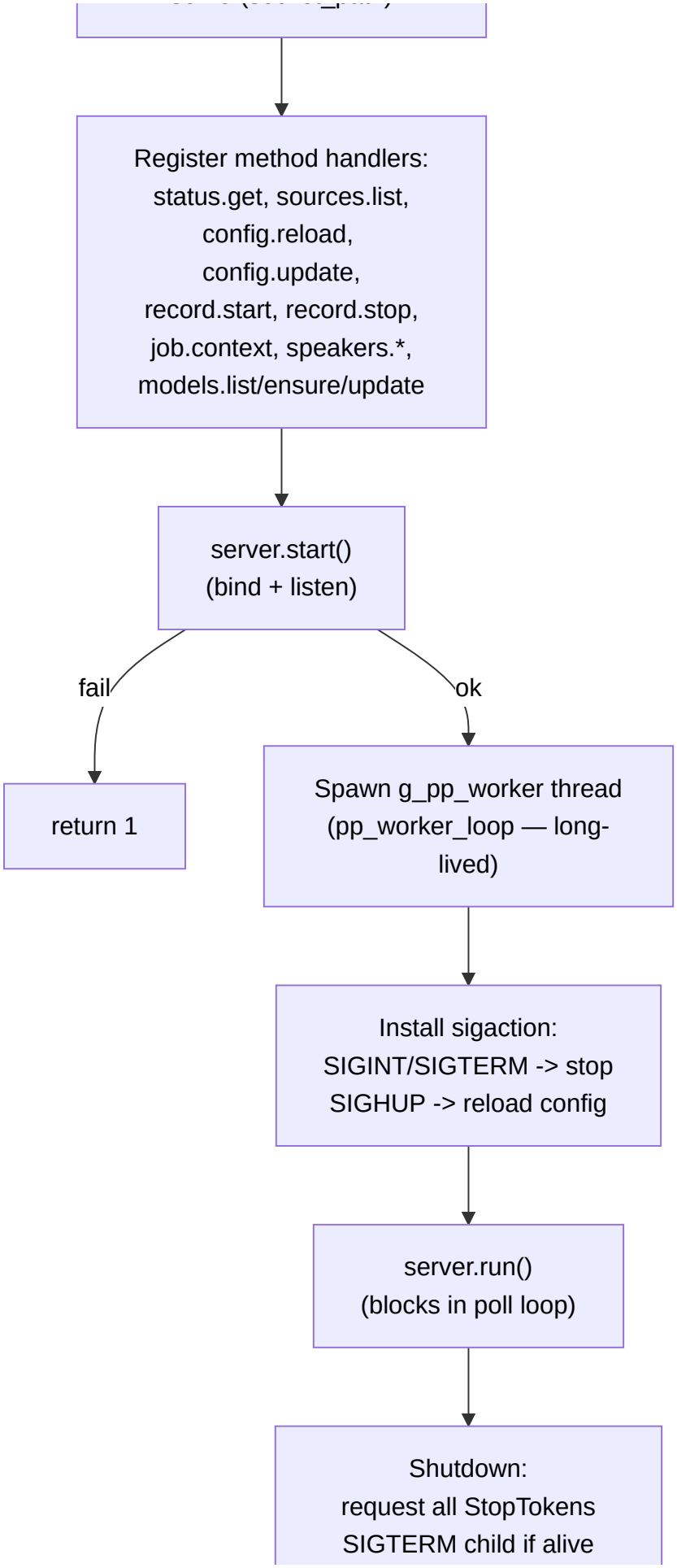
Exact source file inventory per CMake target and the link dependency graph.

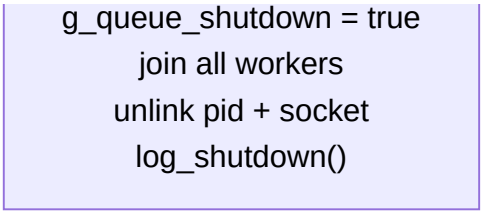


3. Daemon Internals

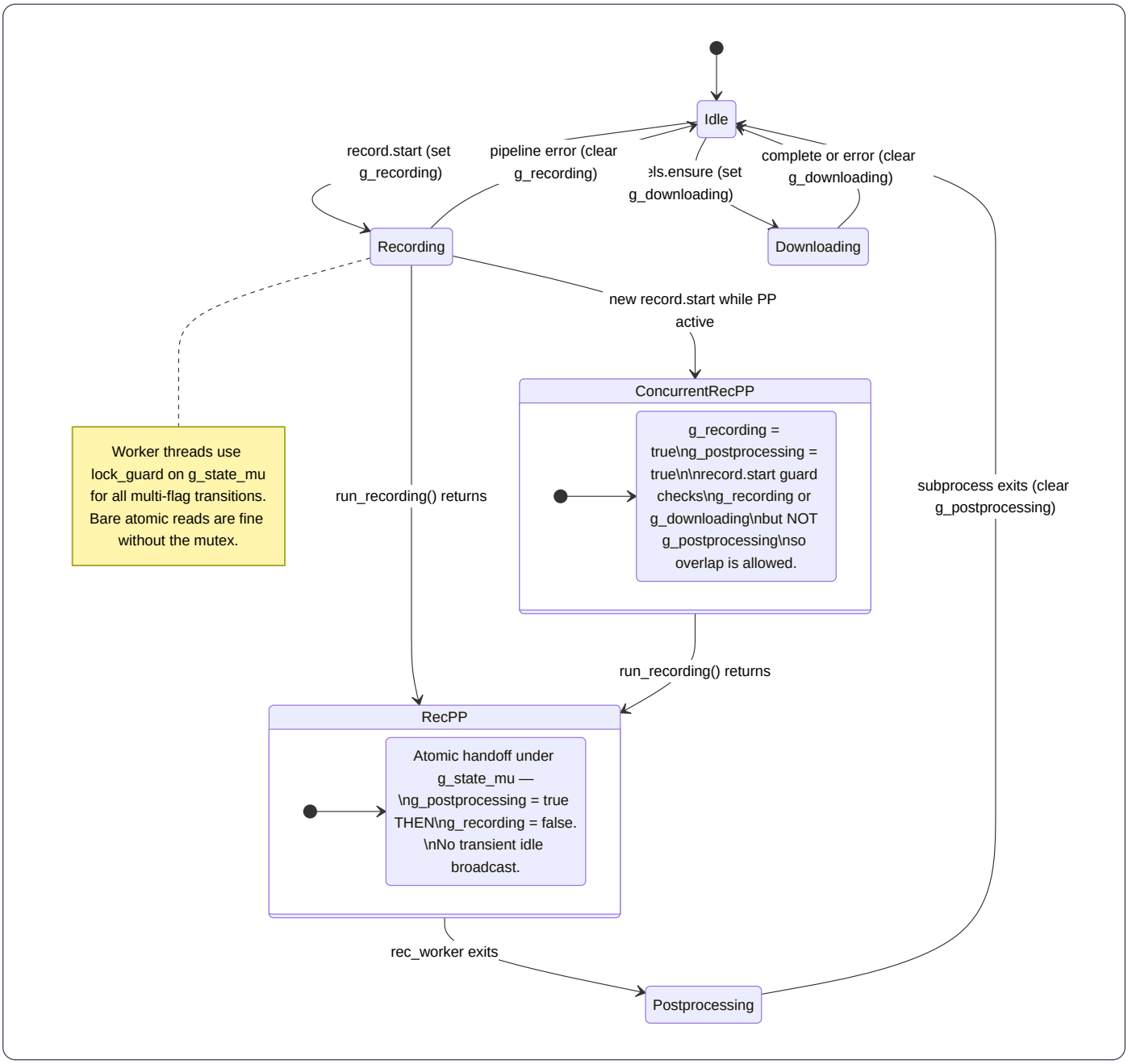
3a. Daemon Startup Sequence


main(argc, argv)
Parse CLI args (--socket, --listen, --log-level)
Compute pid_path Unix: socket.pid TCP: daemon-tcp.pid
open() + flock(LOCK_EX|LOCK_NB)
locked
acquired
stderr: 'Another instance running' return 1
log_init() whisper_log_set(null) load_config() resolve_api_key() notify_init()
Resolve g_self_exe (/proc/self/exe -> sibling 'recmeet')
IpcServer server(socket_path)

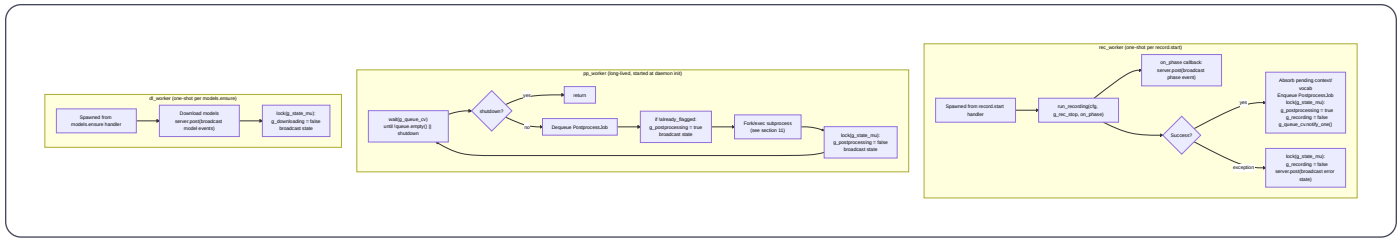

Register method handlers:
status.get, sources.list,
config.reload,
config.update,
record.start, record.stop,
job.context, speakers.*,
models.list/ensure/update
server.start()
(bind + listen)
fail
ok
return 1
Spawn g_pp_worker thread
(pp_worker_loop — long-lived)
Install sigaction:
SIGINT/SIGTERM -> stop
SIGHUP -> reload config
server.run()
(blocks in poll loop)
Shutdown:
request all StopTokens
SIGTERM child if alive



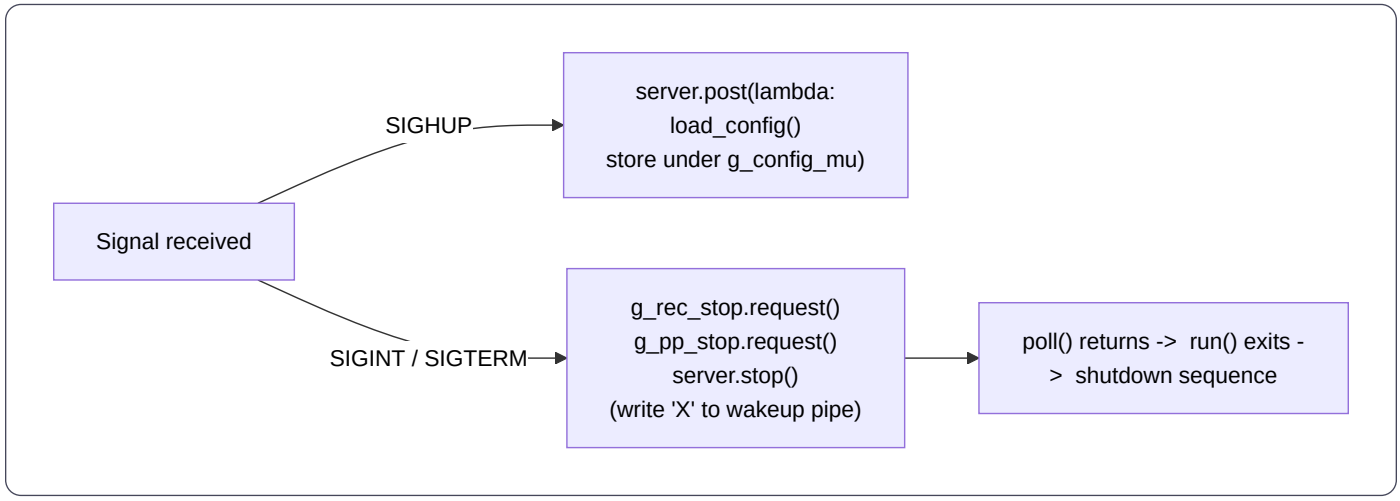
3b. Daemon State Machine



3c. Worker Thread Lifecycle

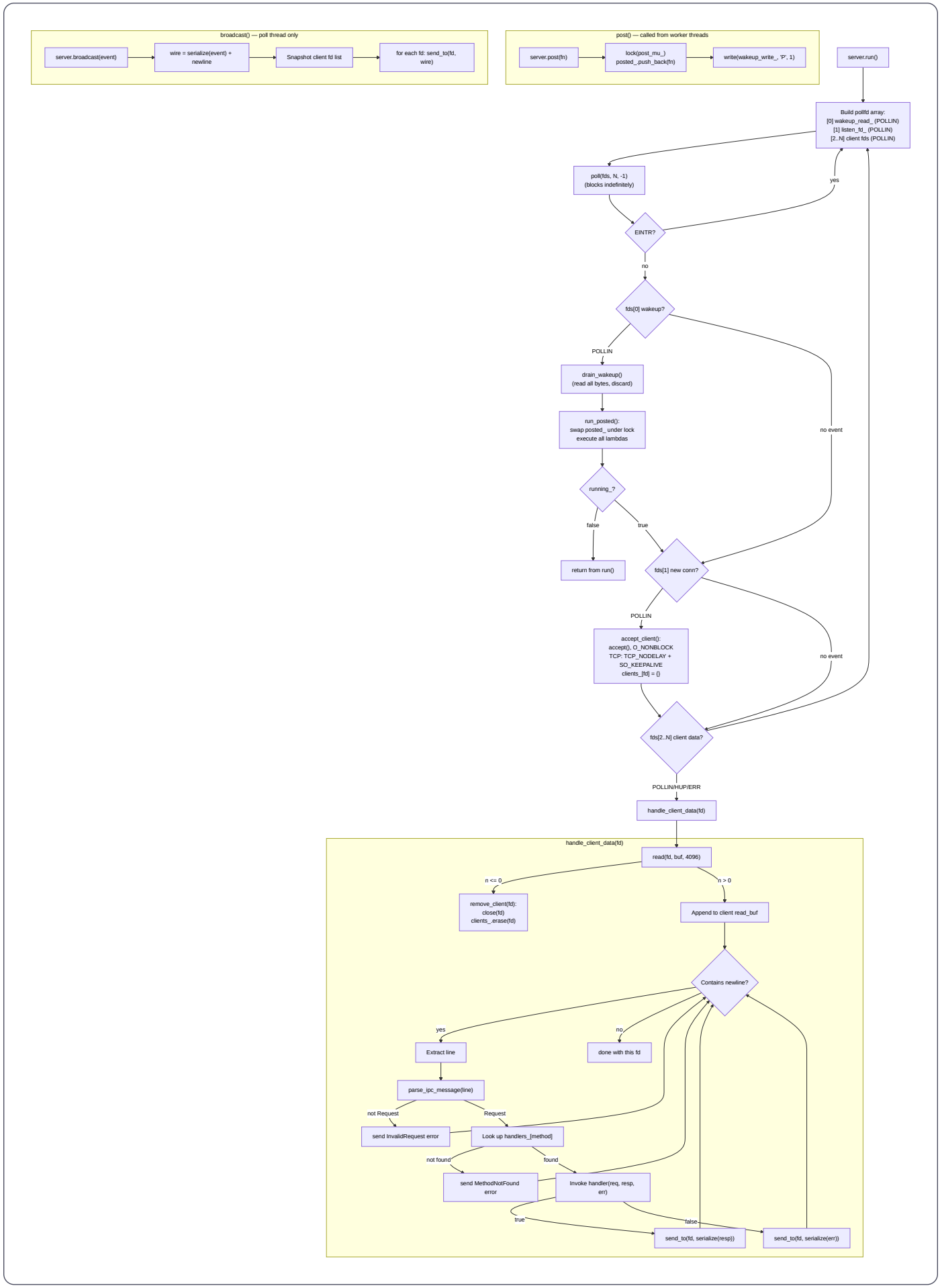


3d. Signal Handling



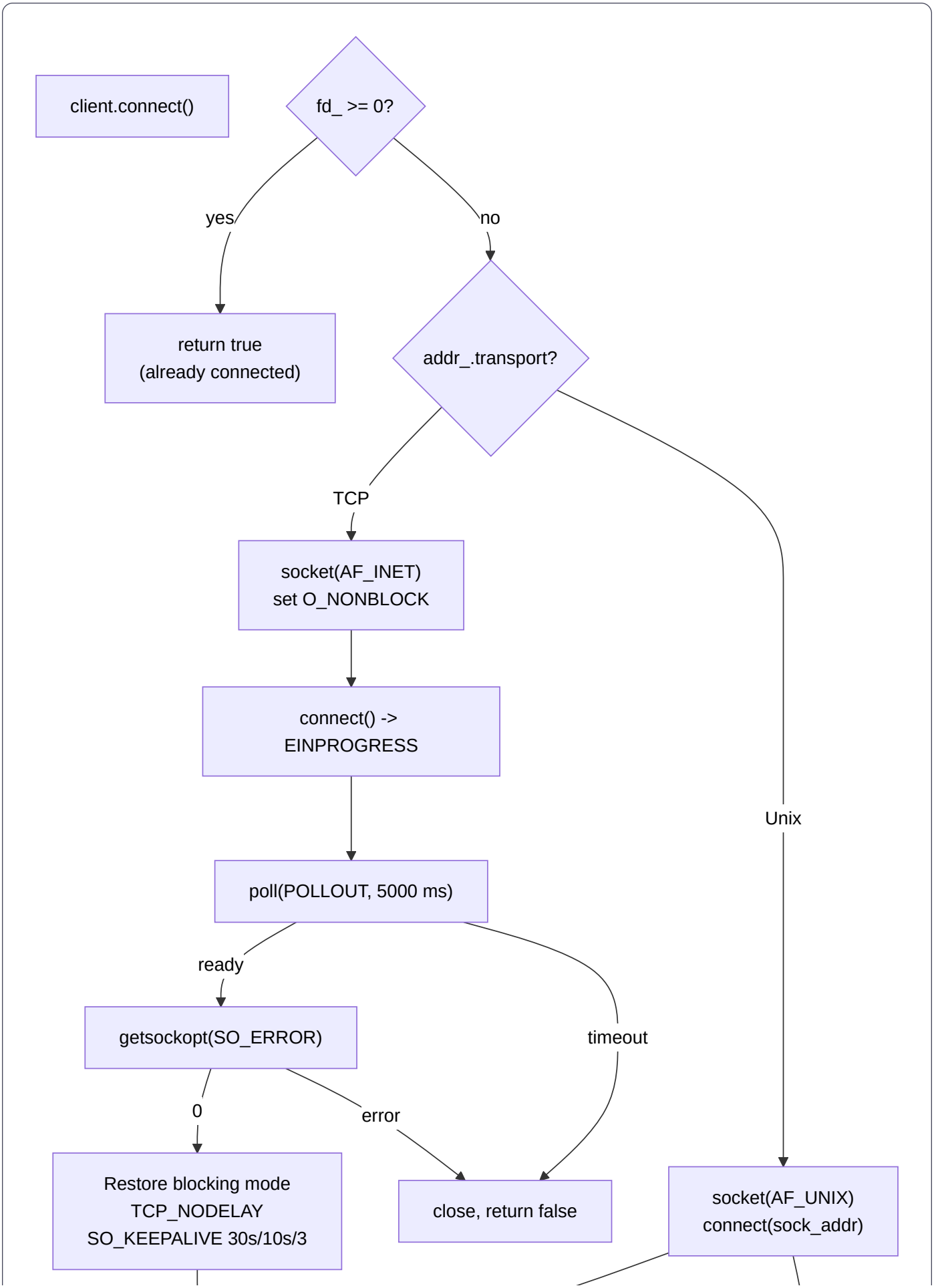
---

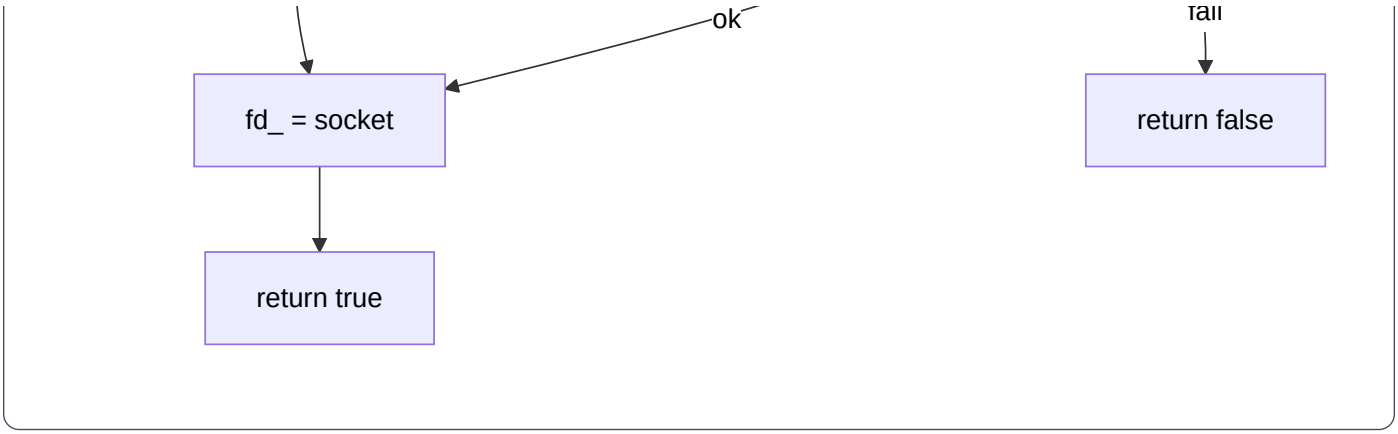
4. IPC Server Poll Loop


broadcast() — poll thread only
server.broadcast(event)
wire = serialize(event) + newline
Snapshot client fd list
for each fd: send_to(fd, wire)
post() — called from worker threads
server.post(fn)
lock(post_mu_) posted_.push_back(fn)
write(wakeup_write_, 'P', 1)
server.run()
Build pollfd array: [0] wakeup_read_ (POLLIN) [1] listen_fd_ (POLLIN) [2..N] client fds (POLLIN)
poll(fds, N, -1) (blocks indefinitely)
yes
EINTR?
no
fds[0] wakeup?
POLLIN
no event
drain_wakeup() (read all bytes, discard)
run_posted(): swap posted_ under lock execute all lambdas
running_?
false
true
return from run()
fds[1] new conn?
POLLIN
no event
accept_client(): accept(), O_NONBLOCK TCP: TCP_NODELAY + SO_KEEPALIVE clients_[fd] = {}
fds[2..N] client data?
POLLIN/HUP/ERR
handle_client_data(fd)
handle_client_data(fd)
read(fd, buf, 4096)
n <= 0
n > 0
remove_client(fd): close(fd) clients_.erase(fd)
Append to client read_buf
Contains newline?
yes
no
Extract line
done with this fd
parse_ipc_message(line)
not Request
Request
send InvalidRequest error
Look up handlers_[method]
not found
found
send MethodNotFound error
Invoke handler(req, resp, err)
true
false
send_to(fd, serialize(resp))
send_to(fd, serialize(err))

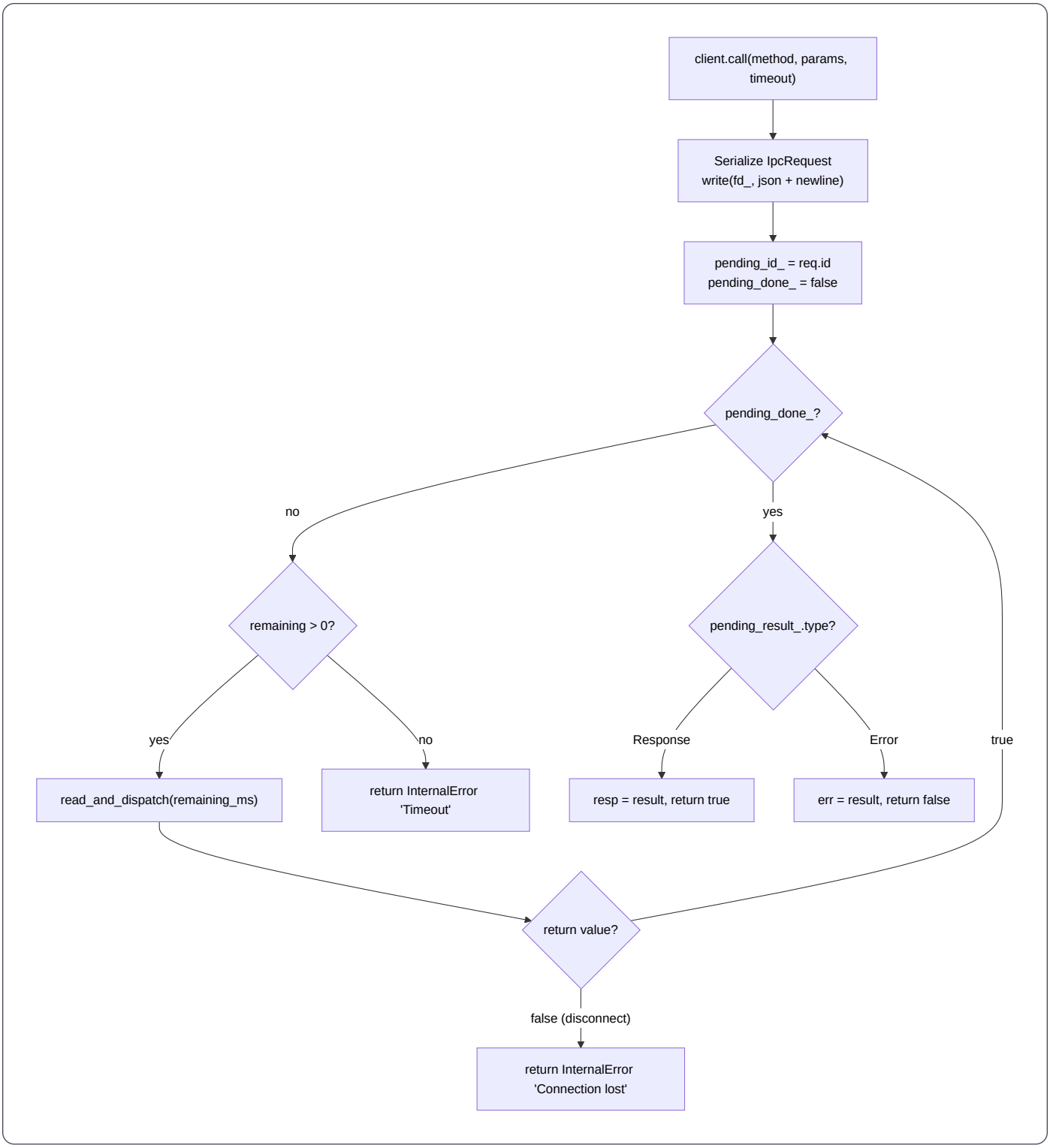
5. IPC Client Flow

5a. Connection

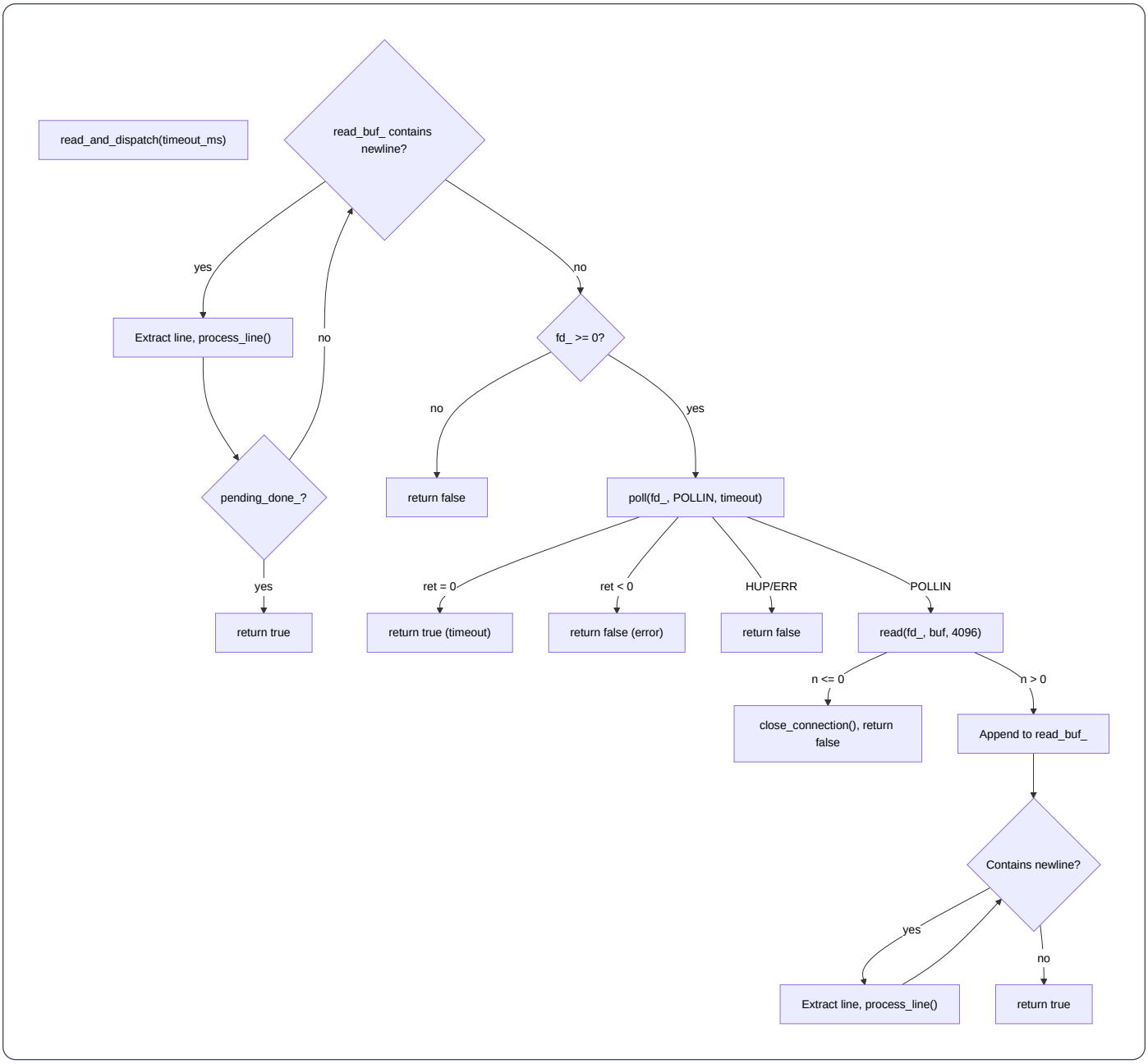

client.connect()
fd_ >= 0?
yes
no
return true
(already connected)
addr_.transport?
TCP
socket(AF_INET)
set O_NONBLOCK
connect() ->
EINPROGRESS
Unix
poll(POLLOUT, 5000 ms)
ready
timeout
getsockopt(SO_ERROR)
0
error
Restore blocking mode
TCP_NODELAY
SO_KEEPALIVE 30s/10s/3
close, return false
socket(AF_UNIX)
connect(sock_addr)



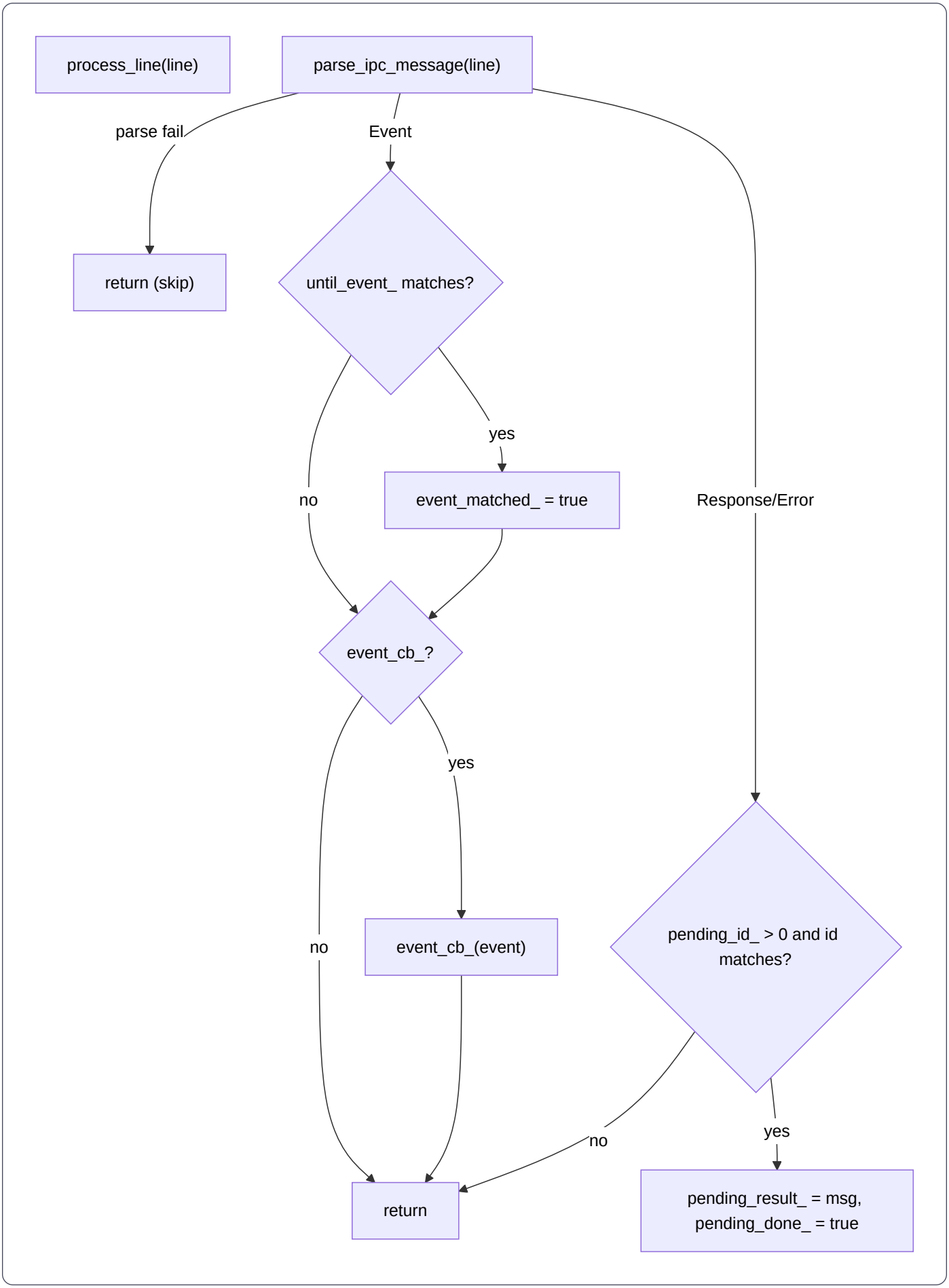
5b. Blocking RPC (call)



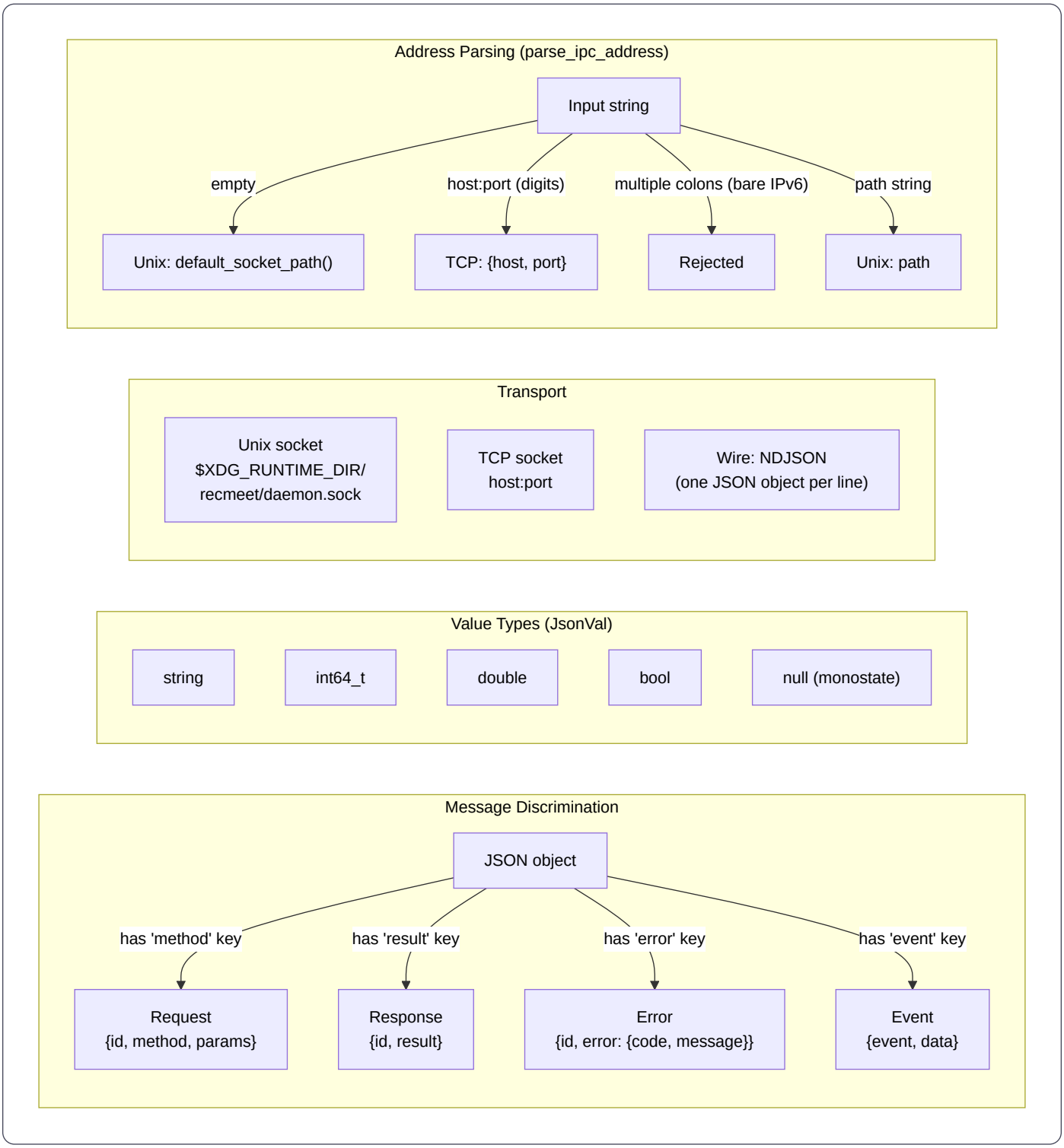
5c. read_and_dispatch() Internal



5d. process_line() Dispatch

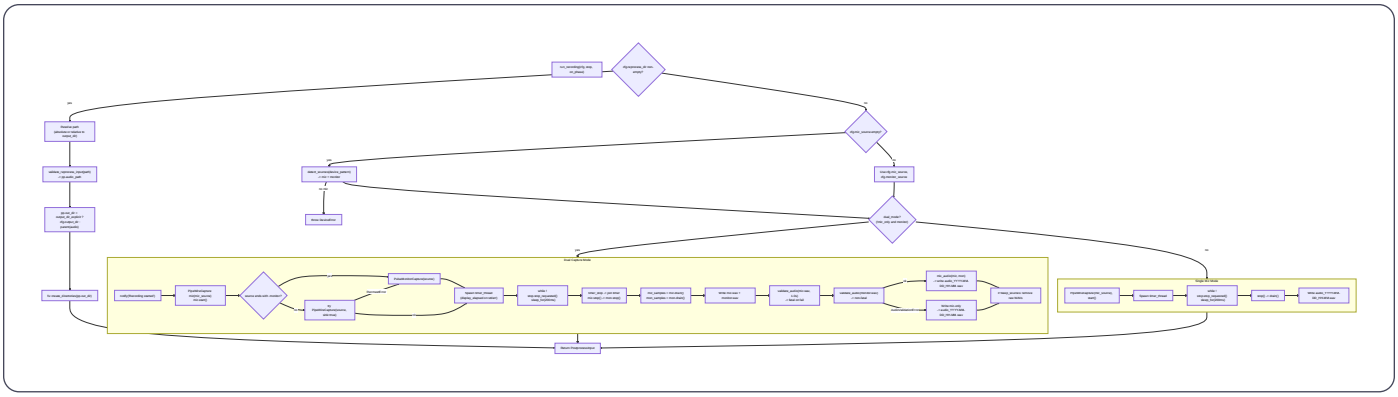

process_line(line)
parse_ipc_message(line)
parse fail
Event
return (skip)
until_event_ matches?
yes
no
event_matched_ = true
Response/Error
event_cb_?
yes
no
event_cb_(event)
pending_id_ > 0 and id matches?
no
yes
return
pending_result_ = msg, pending_done_ = true

6. IPC Protocol Wire Format



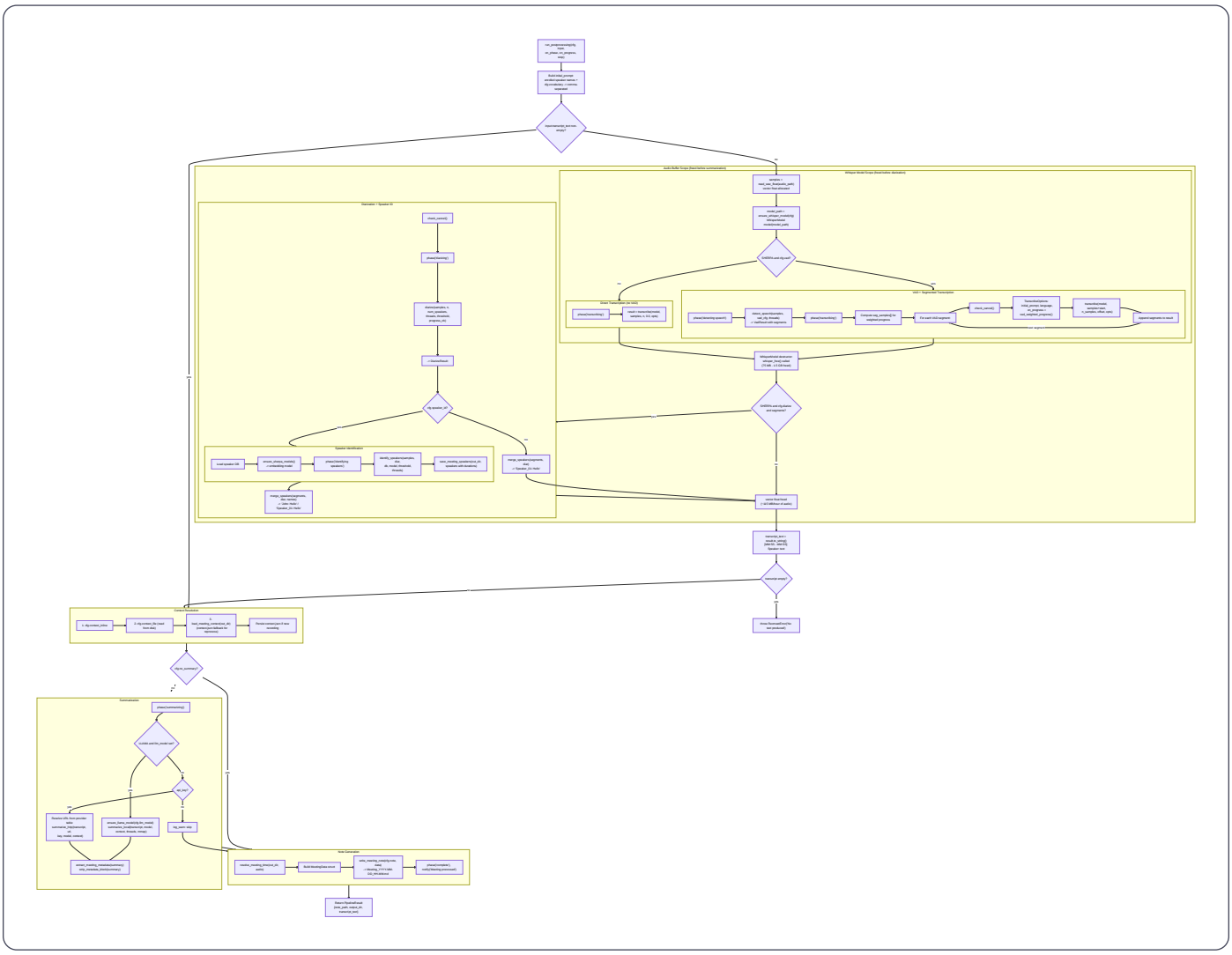
7. Recording Pipeline

`run_recording()` control flow with dual-capture and reprocess branching.



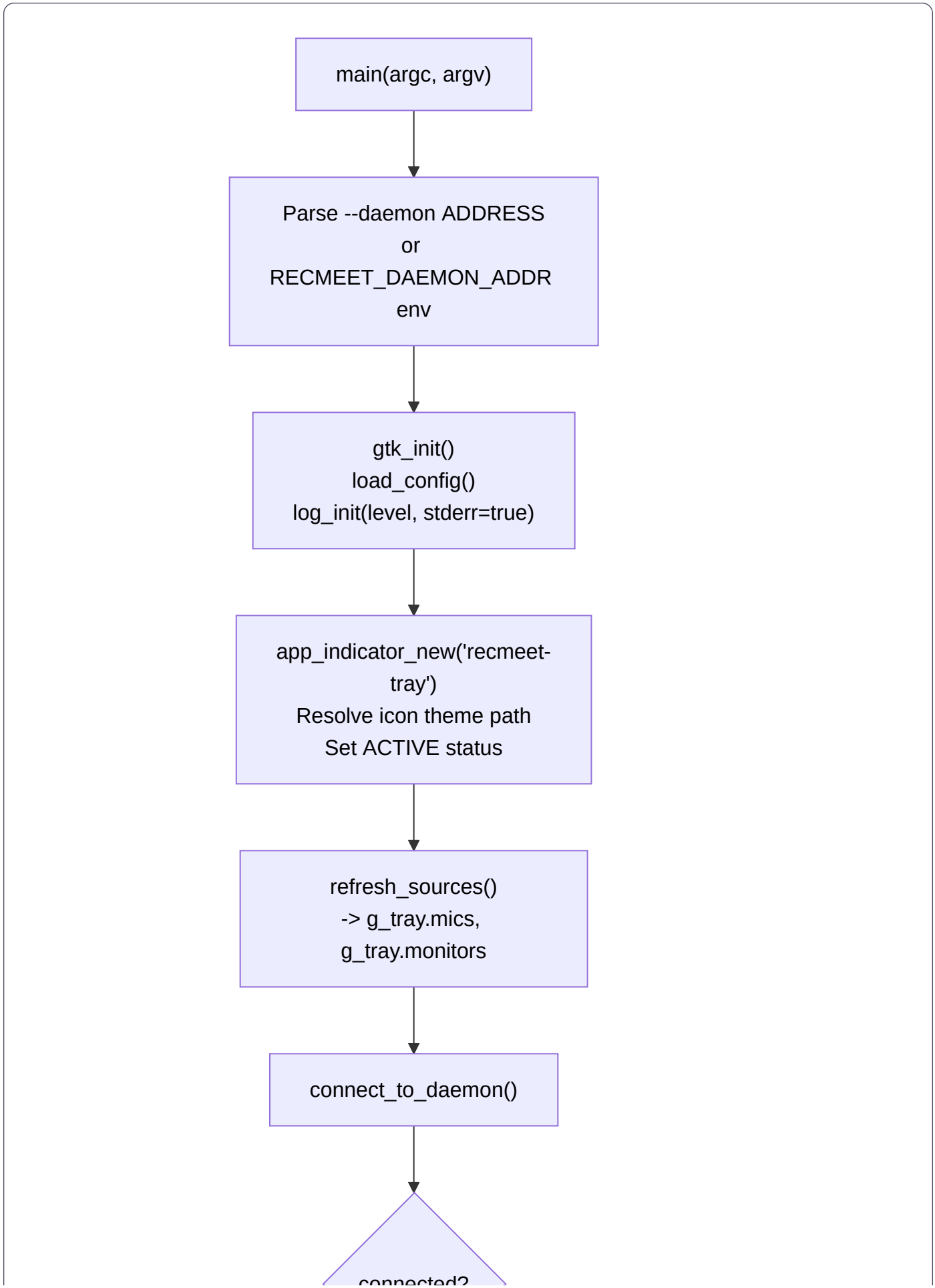
8. Postprocessing Pipeline

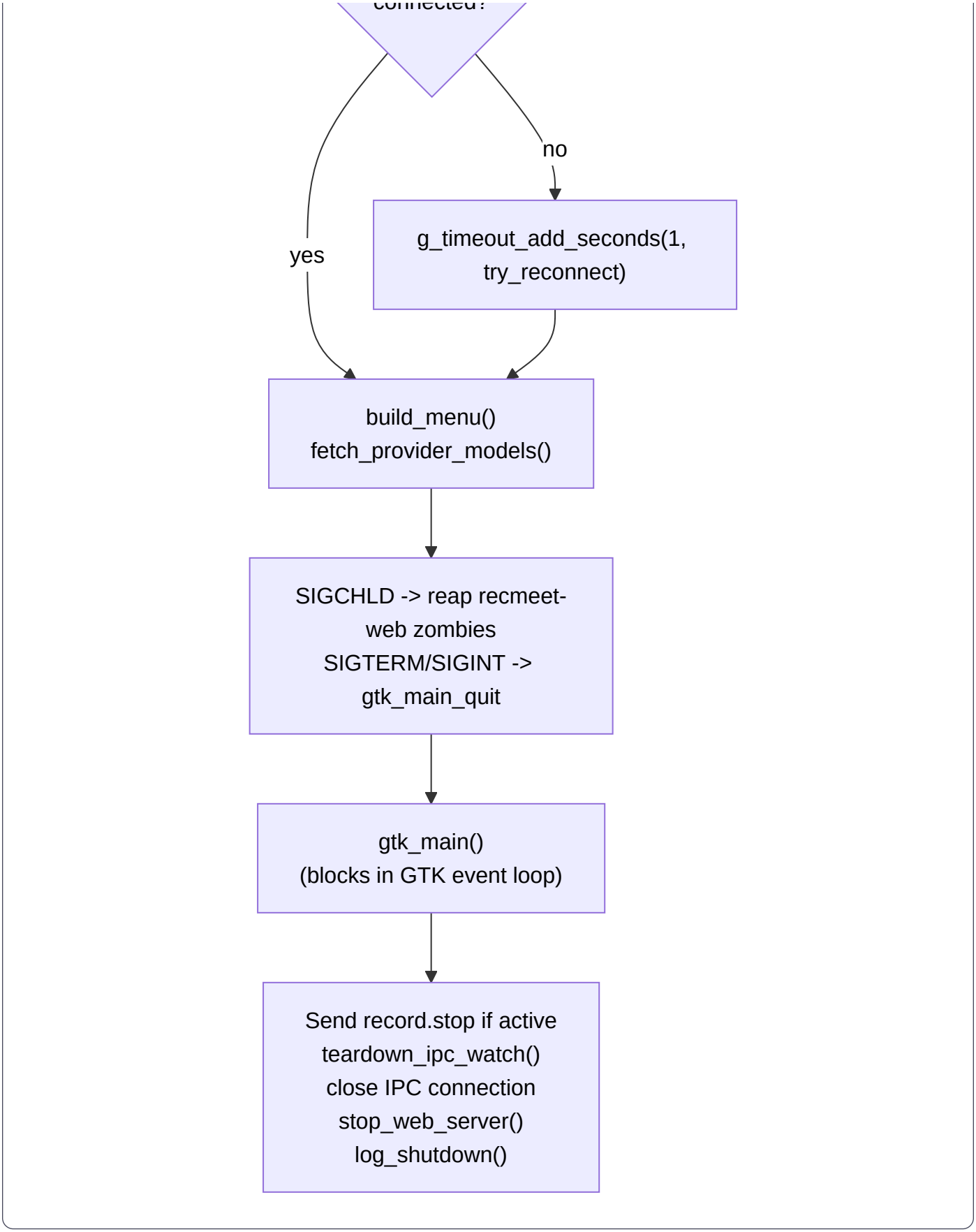
`run_postprocessing()` with memory scoping, cancellation, and all ML stages.



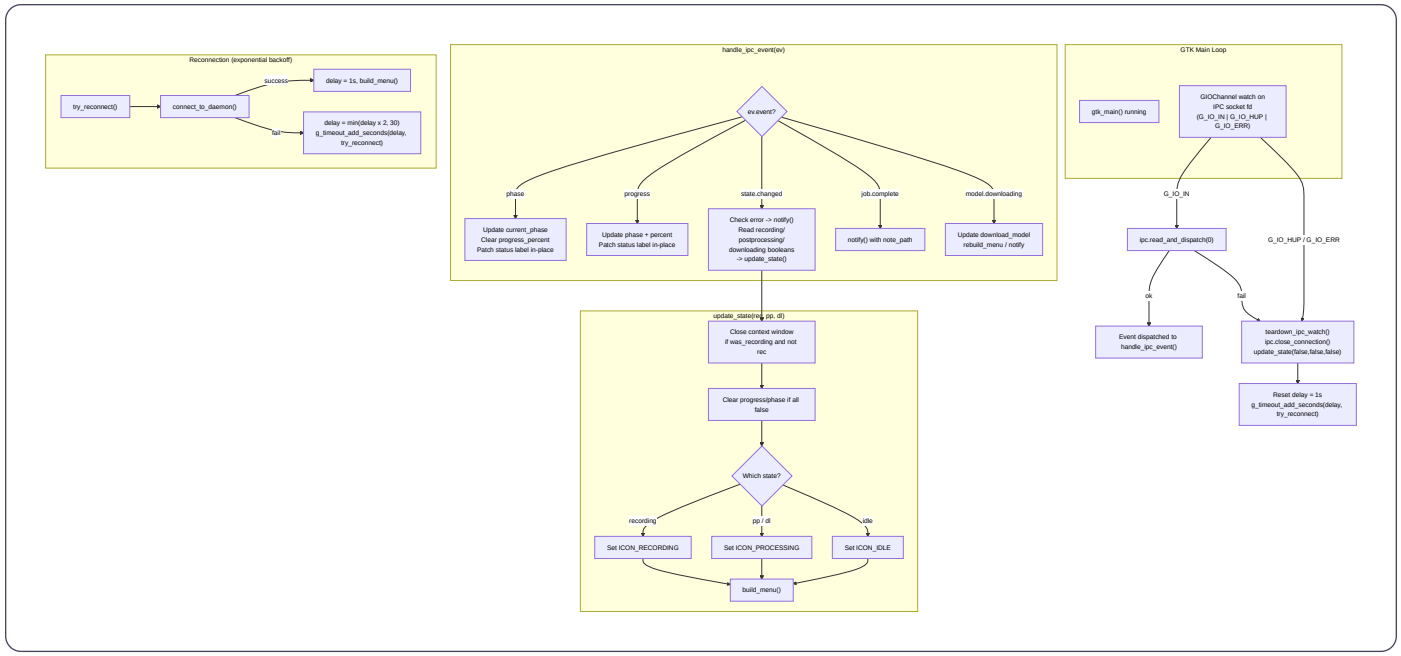
9. Tray Applet

9a. Tray Startup and GTK Main Loop

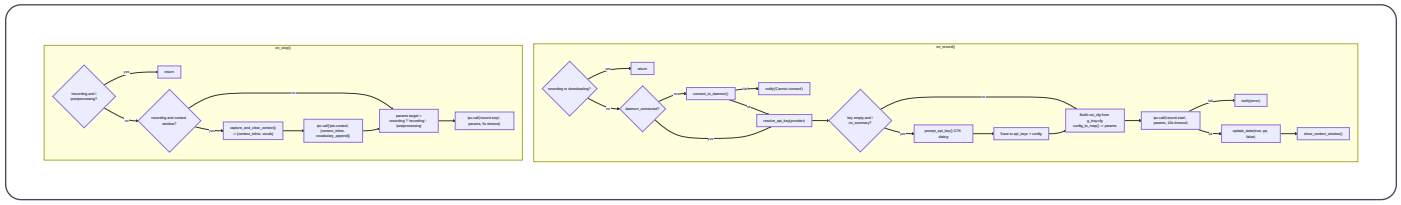

main(argc, argv)
Parse --daemon ADDRESS or RECMEET_DAEMON_ADDR env
gtk_init()
load_config()
log_init(level, stderr=true)
app_indicator_new('recmeet-tray')
Resolve icon theme path
Set ACTIVE status
refresh_sources()
-> g_tray.mics, g_tray.monitors
connect_to_daemon()
connected?



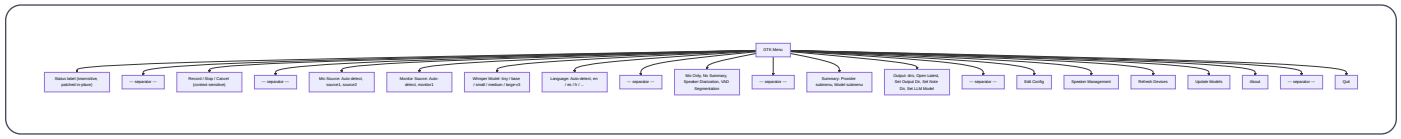
9b. IPC Event Integration with GTK



9c. Recording Start/Stop Flow

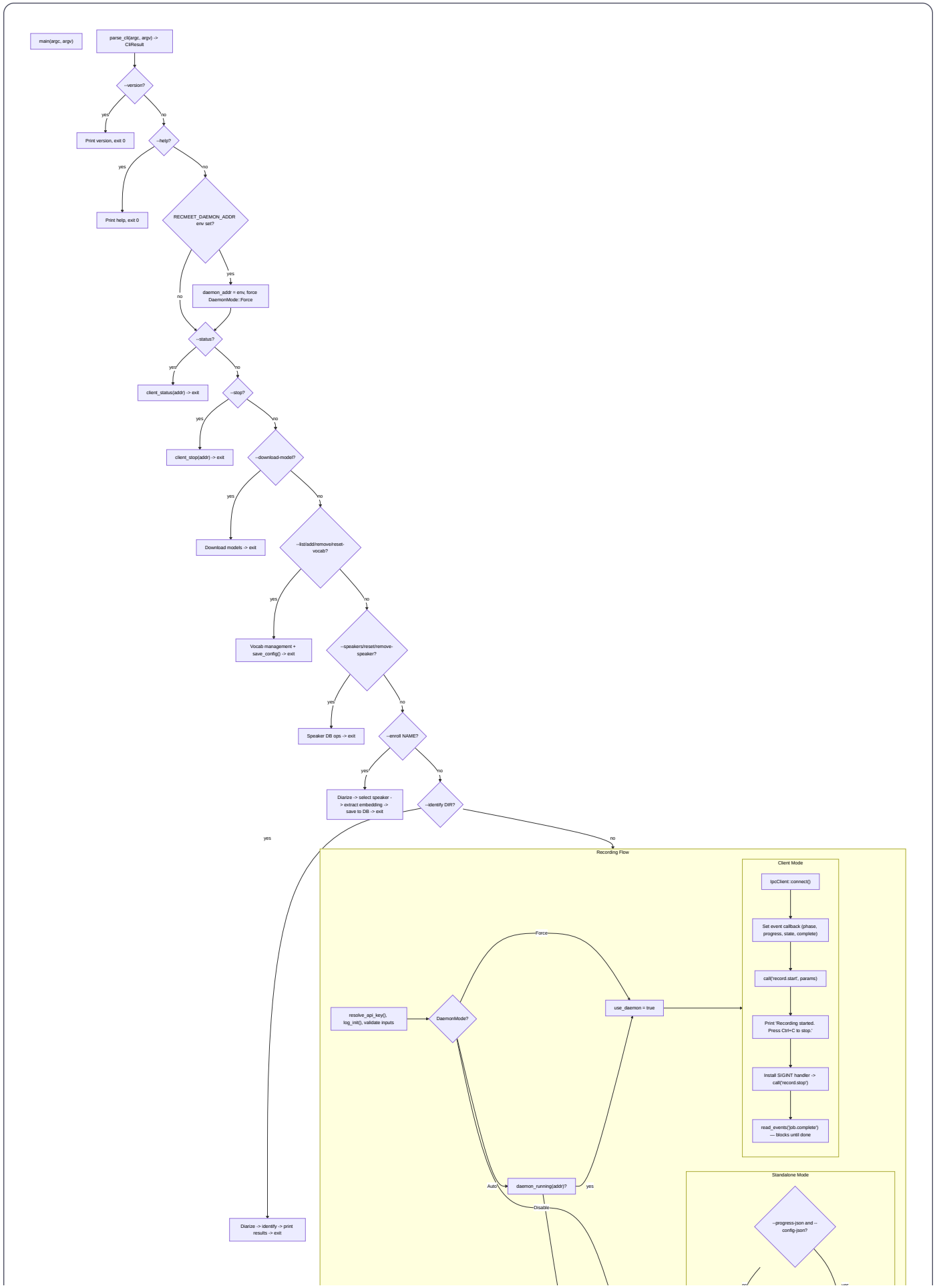


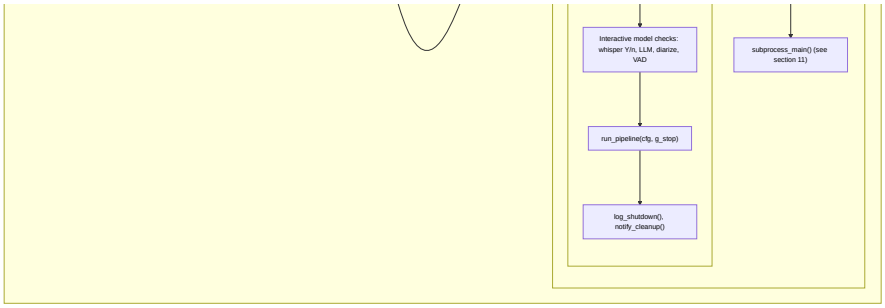
9d. Tray Menu Structure



---

10. CLI Mode Selection

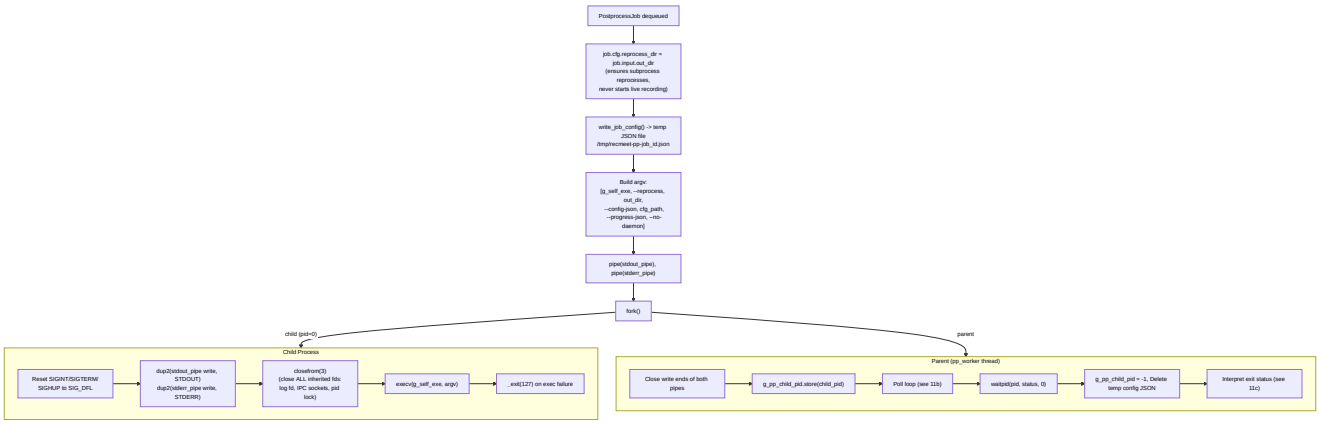

main(argc, argv)
parse_cli(argc, argv) -> CliResult
--version?
yes
no
Print version, exit 0
--help?
yes
no
Print help, exit 0
RECMEET_DAEMON_ADDR env set?
yes
no
daemon_addr = env, force DaemonMode::Force
--status?
yes
no
client_status(addr) -> exit
--stop?
yes
no
client_stop(addr) -> exit
--download-model?
yes
no
Download models -> exit
--list/add/remove/reset-vocab?
yes
no
Vocab management + save_config() -> exit
--speakers/reset/remove-speaker?
yes
no
Speaker DB ops -> exit
--enroll NAME?
yes
no
Diarize -> select speaker -> extract embedding -> save to DB -> exit
--identify DIR?
yes
no
Diarize -> identify -> print results -> exit
Recording Flow
resolve_api_key(), log_init(), validate inputs
DaemonMode?
Force
Auto
Disable
daemon_running(addr)?
yes
use_daemon = true
Client Mode
IpcClient::connect()
Set event callback (phase, progress, state, complete)
call("record.start", params)
Print 'Recording started. Press Ctrl+C to stop.'
Install SIGINT handler -> call("record.stop")
read_events("job.complete") — blocks until done
Standalone Mode
--progress-json and --config-json?



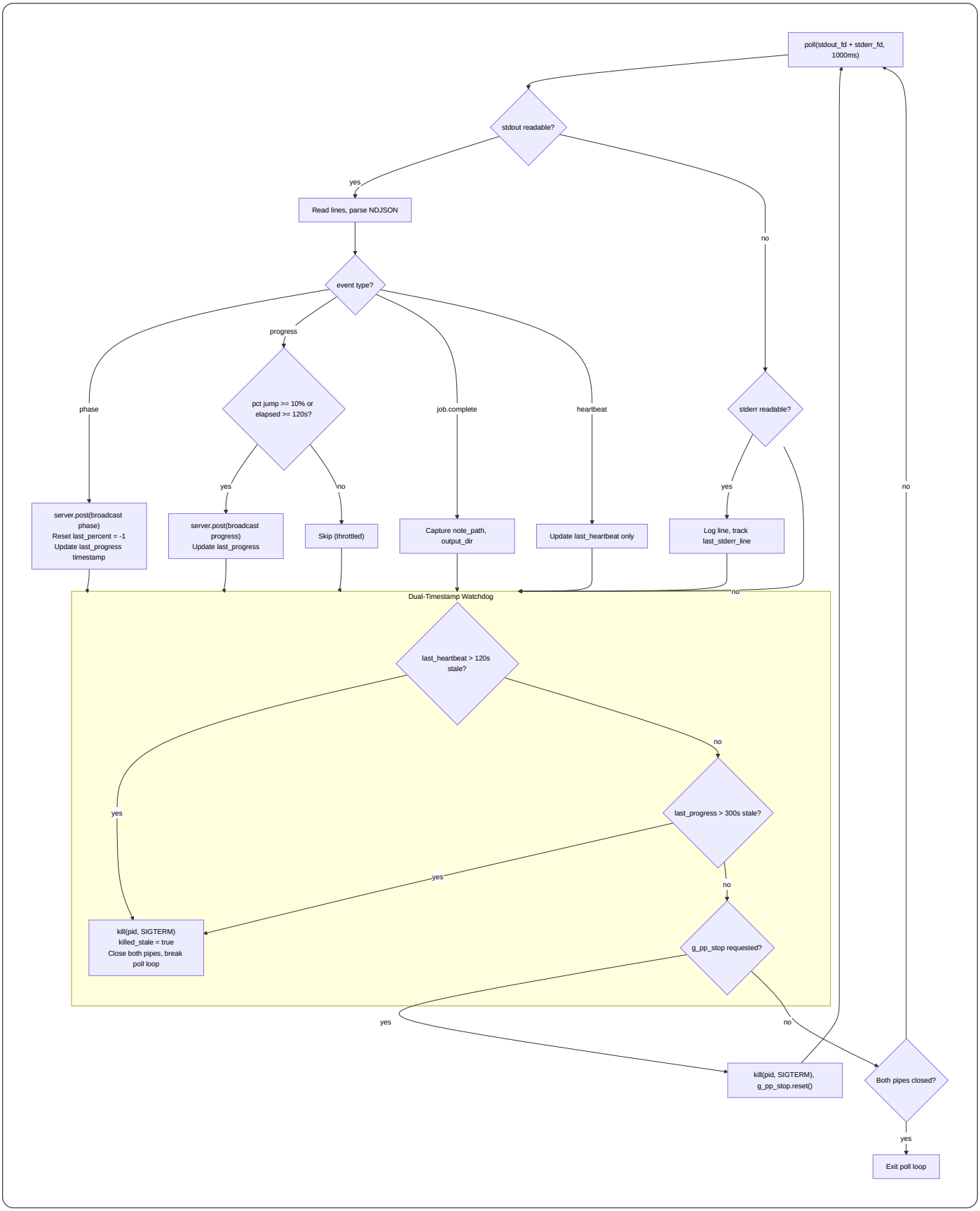
11. Subprocess Postprocessing

The daemon's `pp_worker_loop` fork/exec's the `recmeet` binary as a child process for crash isolation. Communication is via NDJSON on stdout/stderr pipes.

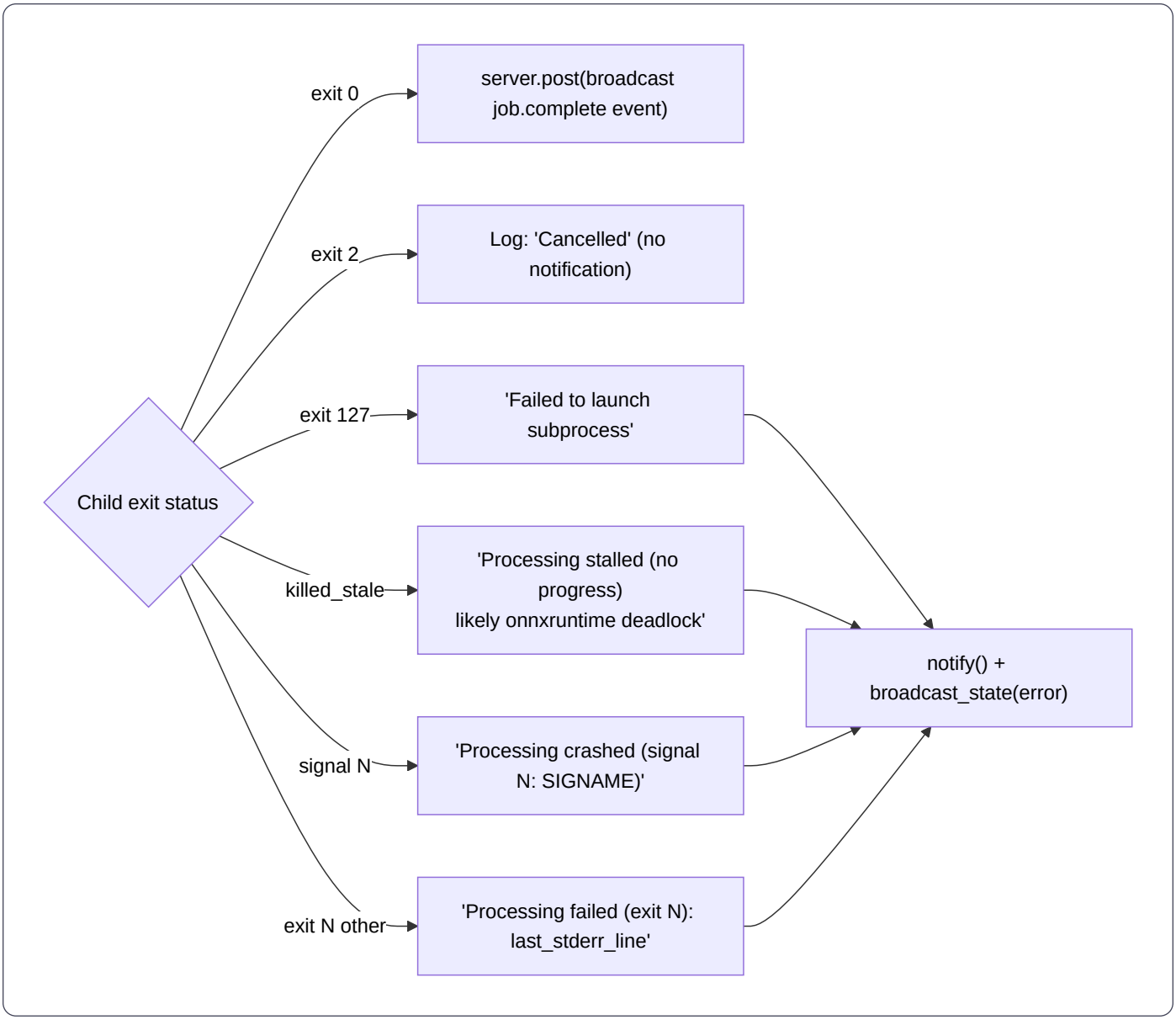
11a. Fork/Exec Flow



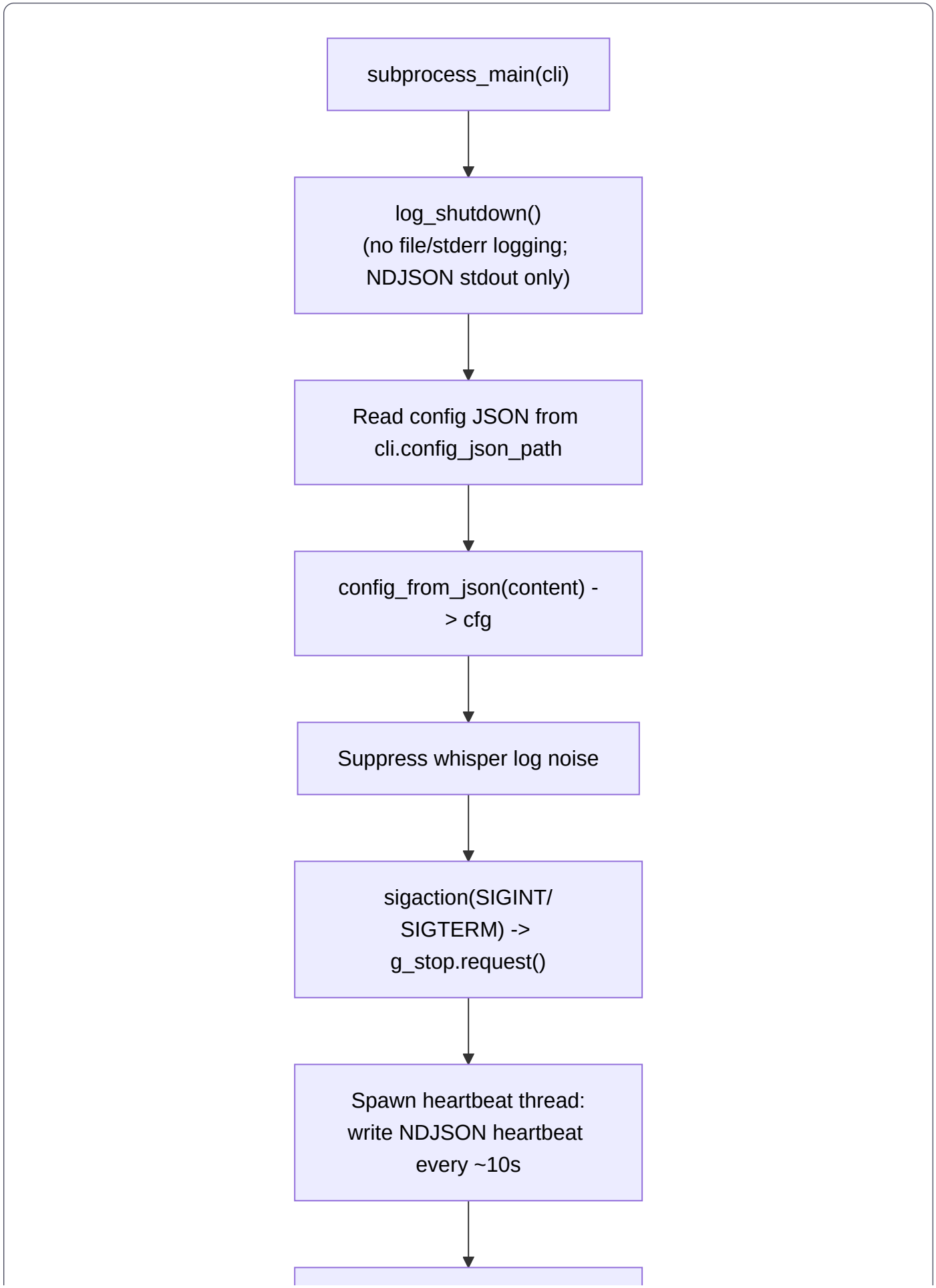
11b. NDJSON Poll Loop and Watchdog

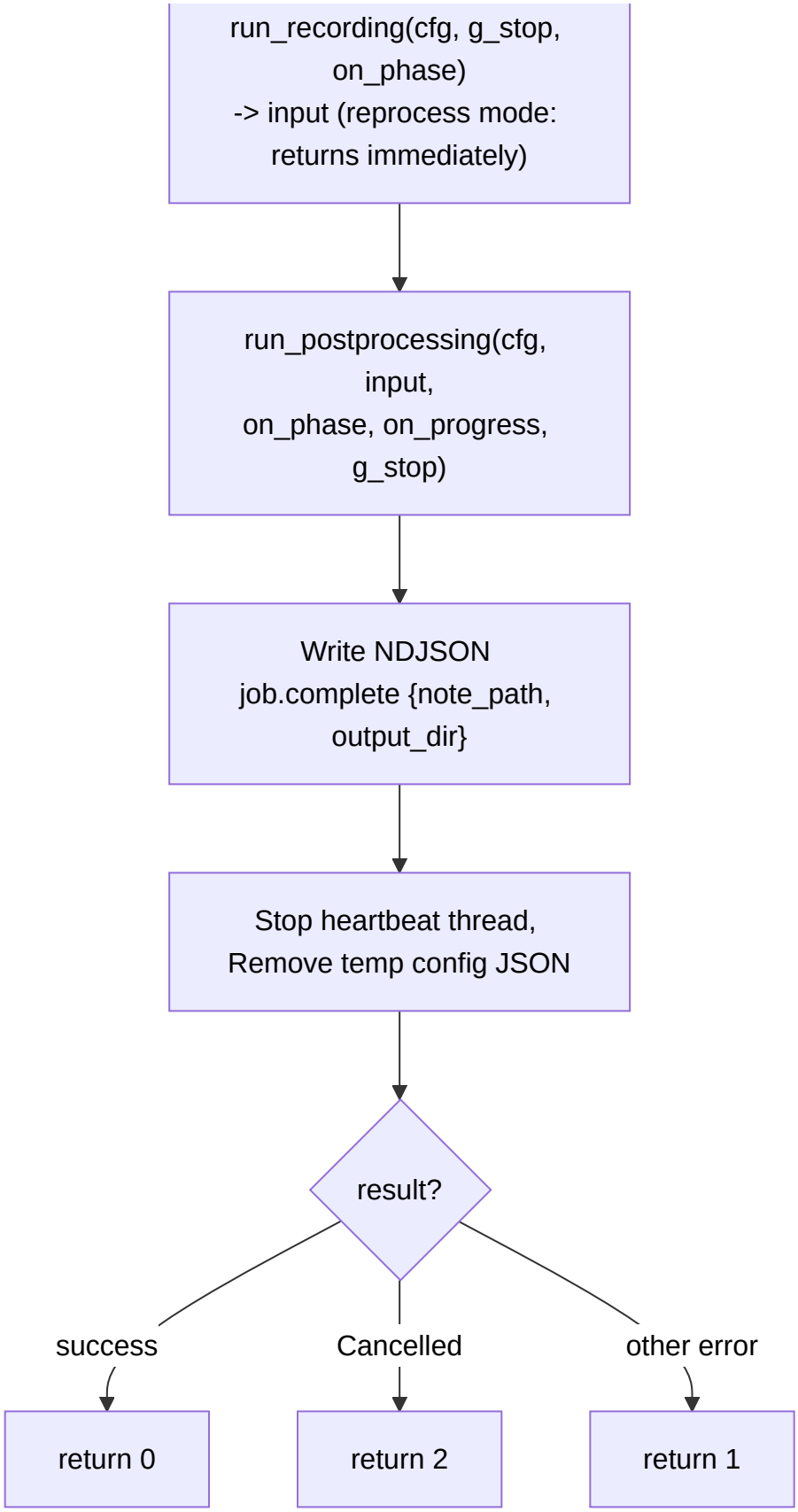


11c. Exit Status Interpretation

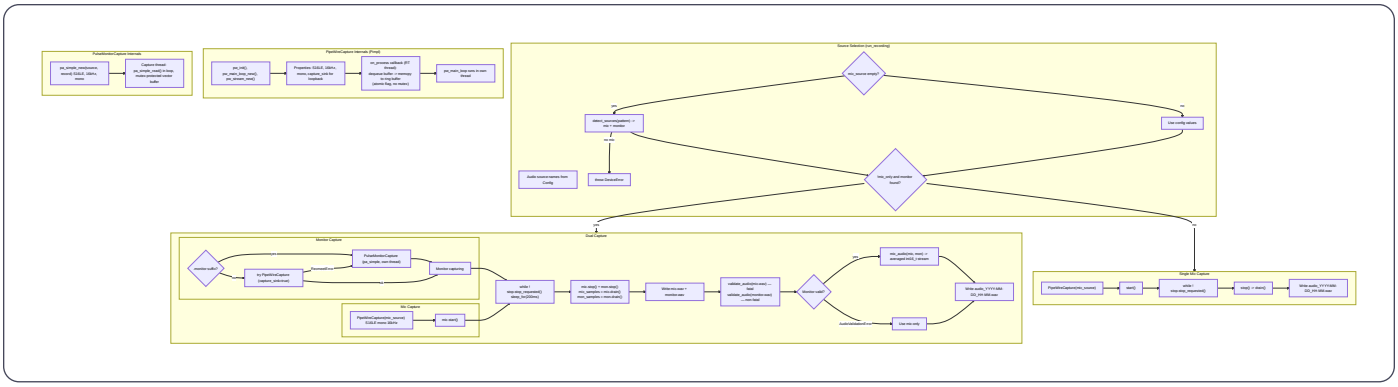


11d. Subprocess Internal Flow


subprocess_main(cli)
log_shutdown()
(no file/stderr logging;
NDJSON stdout only)
Read config JSON from
cli.config_json_path
config_from_json(content) -
> cfg
Suppress whisper log noise
sigaction(SIGINT/
SIGTERM) ->
g_stop.request()
Spawn heartbeat thread:
write NDJSON heartbeat
every ~10s



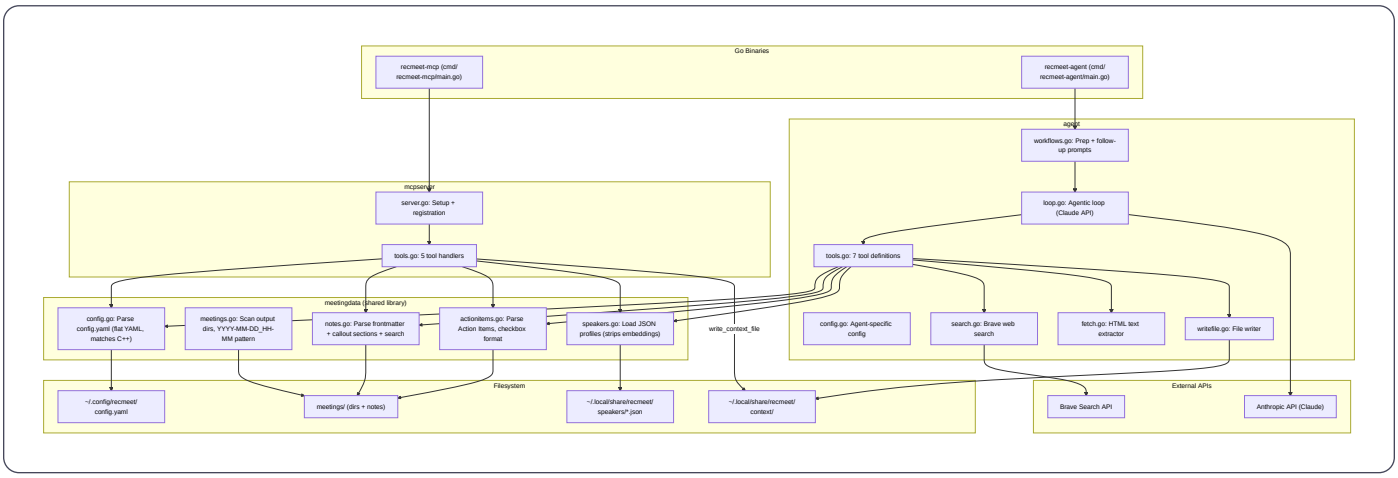
12. Audio Capture Subsystem



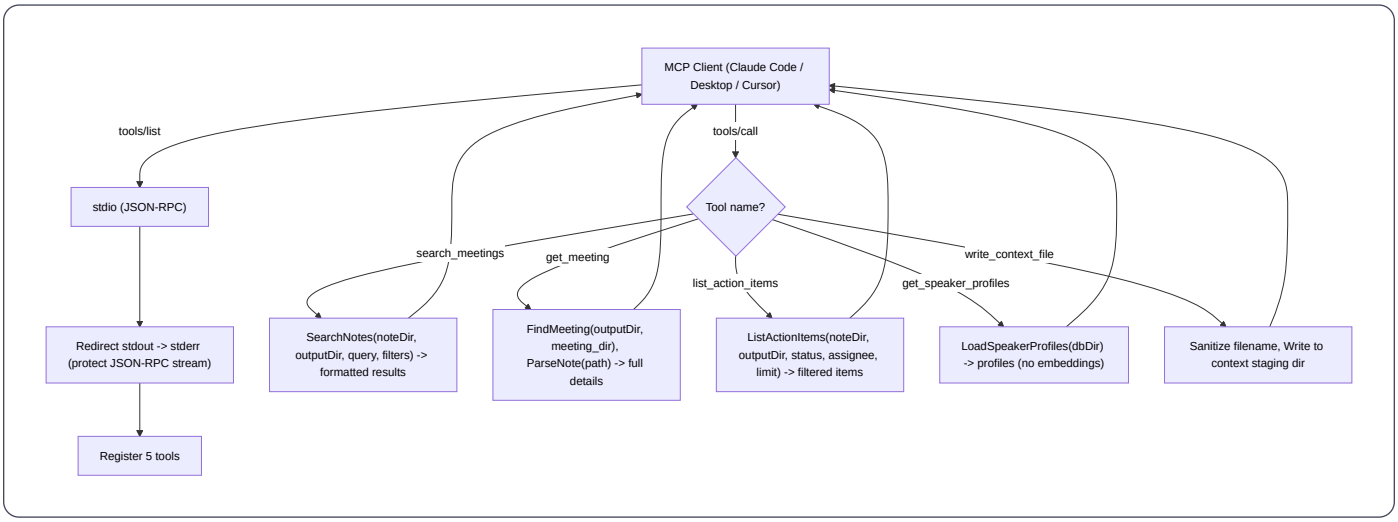
---

13. Go Tools Module

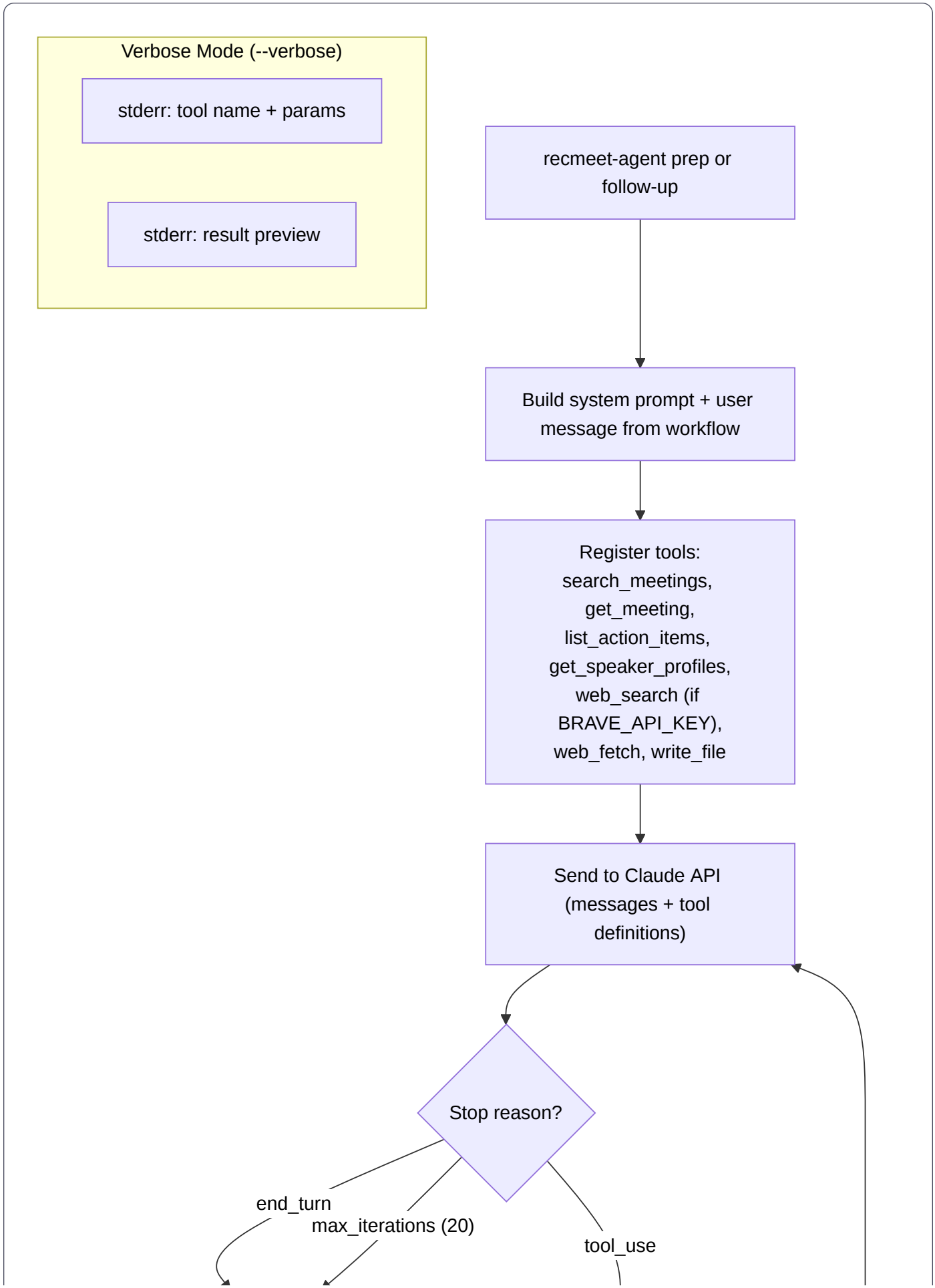
13a. Module Structure and Data Flow

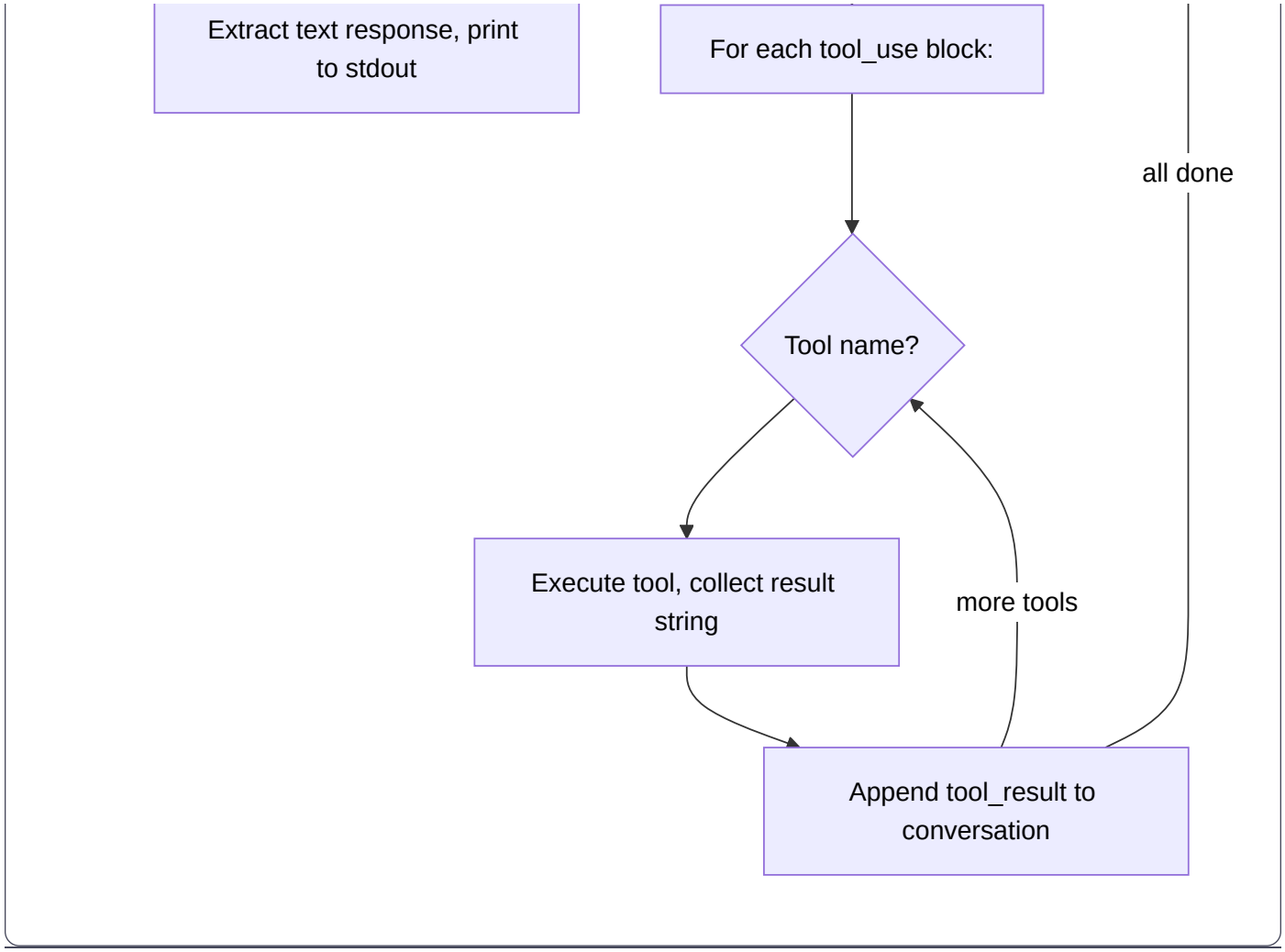


13b. MCP Server Tool Dispatch



13c. Agent Agentic Loop


Verbose Mode (--verbose)
stderr: tool name + params
stderr: result preview
recmeet-agent prep or follow-up
Build system prompt + user message from workflow
Register tools: search_meetings, get_meeting, list_action_items, get_speaker_profiles, web_search (if BRAVE_API_KEY), web_fetch, write_file
Send to Claude API (messages + tool definitions)
Stop reason?
end_turn
max_iterations (20)
tool_use



Generated from `docs/COMPONENT-DIAGRAMS.md` — recmeet project